Лабораторная работа №8

Архитектура ЭВМ

Альманасра Рами

Content

Цель работы **5**

Задание **6**

Теоретическое введение **7**

Выполнение лабораторной работы **8**

Реализация циклов в NASM 8

Обработка аргументов командной строки 11

Задание для самостоятельной работы 15

Выводы **19**

Список литературы **20**

Список иллюстраций

1 файл lab8-1.asm 8
2 программа из листинга 9
3 запуск программы 9
4 Изменение текст программы 10
5 запуск новой программы 10
6 добавление команды push и pop 11
7 запуск новой программы 11
8 программа из листинга 12
9 запуск программы 12
10 программа из третьего листинга 13
11 запуск программы 13
12 Изменение текст программы 14
13 запуск новой программы 15
14 программа, находящая сумму значений для функции f(x) = 10x-5 . 16
15 запуск программы 18

Список таблиц

Цель работы

Приобретение навыков написания программ с использованием циклов и обработкой аргументов командной строки.

Задание

1. Реализация цикла в NASM
2. Обработка аргументов командной строки
3. Самостоятельное написание программы по материалам лабораторной работы

Теоретическое введение

Стек — это структура данных, организованная по принципу LIFO («Last In — First Out» или «последним пришёл — первым ушёл»). Стек является частью архитектуры процессора и реализован на аппаратном уровне. Для работы со стеком в процессоре есть специальные регистры (ss, bp, sp) и команды.

Основной функцией стека является функция сохранения адресов возврата и передачи аргументов при вызове процедур. Кроме того, в нём выделяется память для локальных переменных и могут временно храниться значения регистров.

Выполнение лабораторной работы

Реализация циклов в NASM

Создаю файл lab8-1.asm в каталог для программам лабораторной работы № 8 (Fig. -@fig:001).

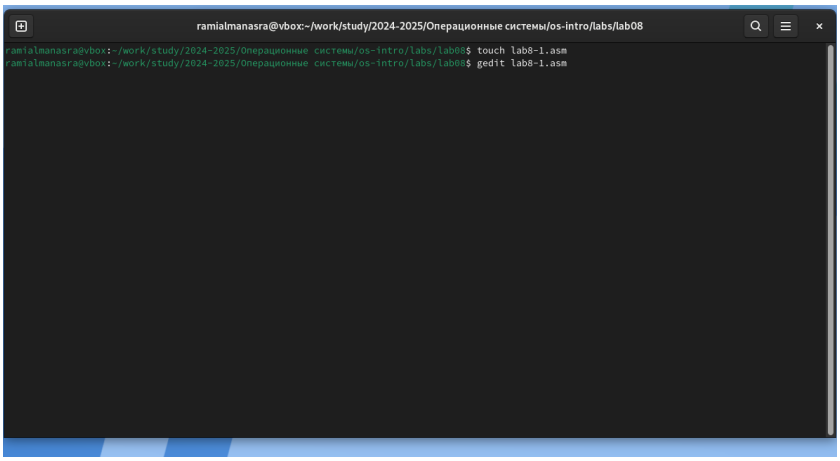

Рис. 1: файл lab8-1.asm

Копирую программу из листинга в созданный файл (Fig. -@fig:002).

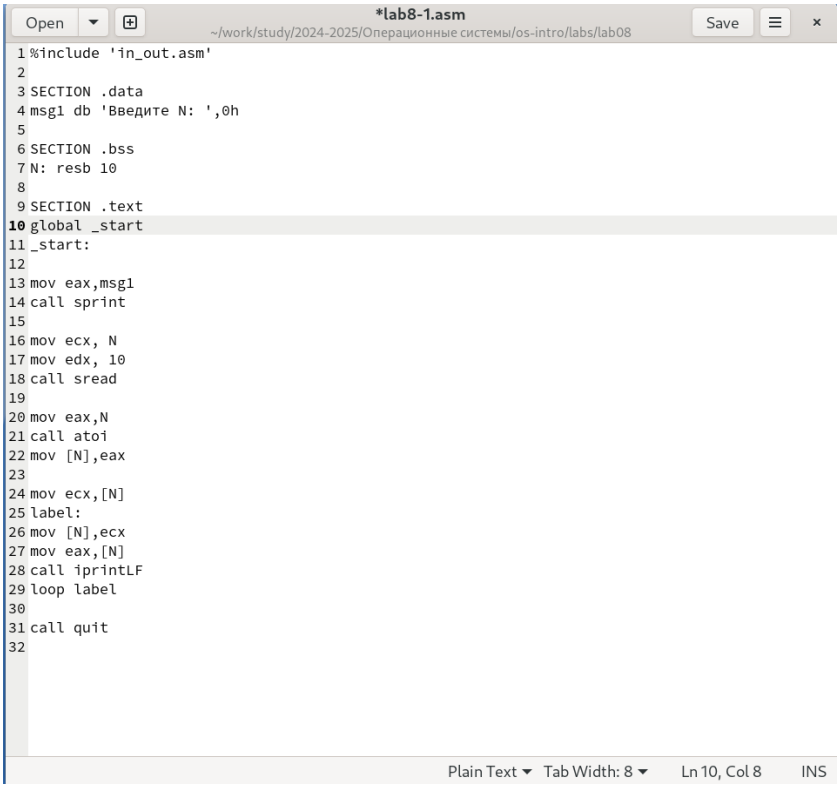

Рис. 2: программа из листинга

Я запускаю программу; она показывает работу циклов в NASM (Fig. -@fig:003).

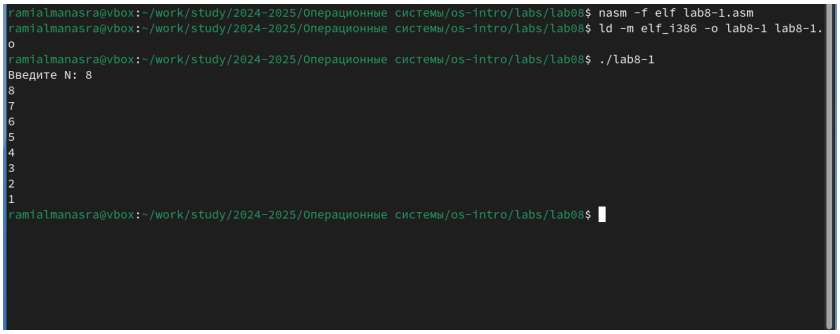

Рис. 3: запуск программы

Изменяю текст программы добавив изменение значение регистра ecx в цикле (Fig. -@fig:004).

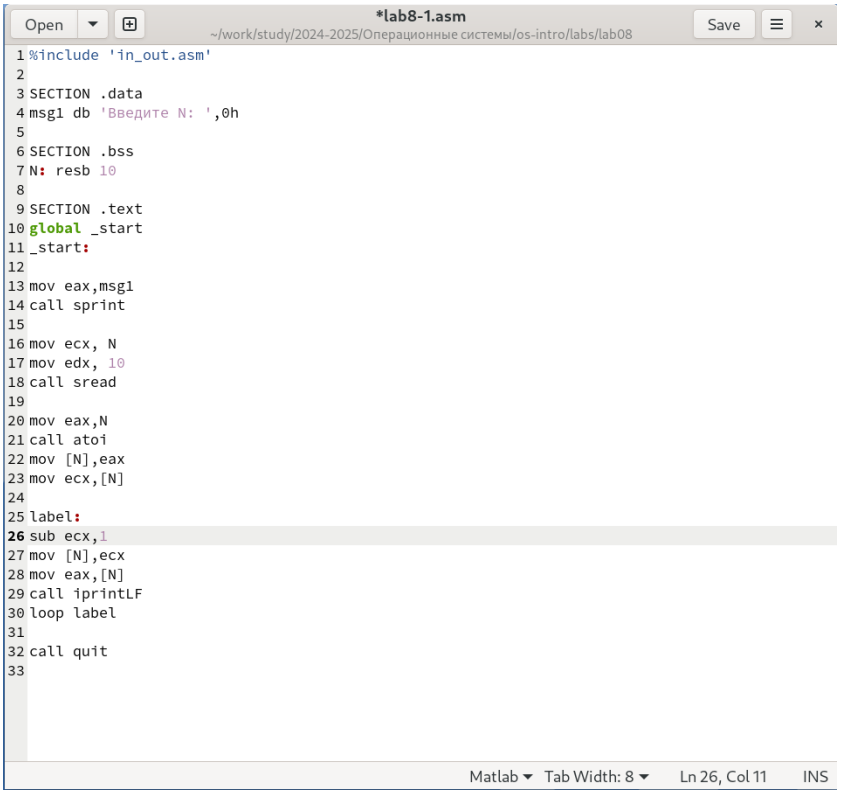

Рис. 4: Изменение текст программы

Из-за того, что теперь значения регистр ecx уменьшается на каждой итерации на 2 , количество итераций сокращается вдвое (Fig. -@fig:005).

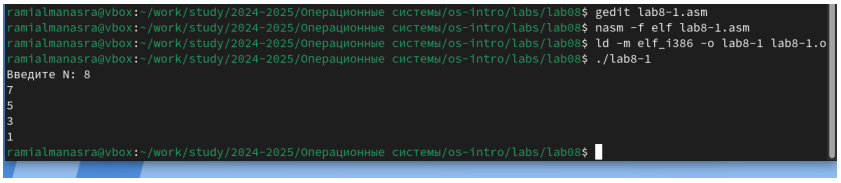

Рис. 5: запуск новой программы

Вношу изменения в текст программы добавив команды push и pop (добавления в стек и извлечения из стека) (Fig. -@fig:006).

```
%include 'in_out.asm'

SECTION .data
msg1 db 'Введите N: ',0h

SECTION .bss
N: resb 10

SECTION .text
global _start
_start:

mov eax,msg1
call sprint

mov ecx, N
mov edx, 10
call sread

mov eax,N
call atoi
mov [N],eax
mov ecx,[N]

label:
push ecx
sub ecx,1
mov [N],ecx
mov eax,[N]
call iprintLF
pop ecx
loop label

call quit
```

Рис. 6: добавление команды push и pop

Теперь количество итераций соответствует введенному N, но выводные числа изменились на -1 (Fig. -@fig:007).

```
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab08$ gedit lab8-1.asm
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab08$ nasm -f elf lab8-1.asm
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab08$ ld -m elf_1386 -o lab8-1 lab8-1.o
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab08$ ./lab8-1
Введите N: 8
7
6
5
4
3
2
1
0
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab08$
```

Рис. 7: запуск новой программы

Обработка аргументов командной строки

Создаю файл lab8-2.asm в каталоге и ввожу в него текст программы из листинга (Fig. -@fig:008).

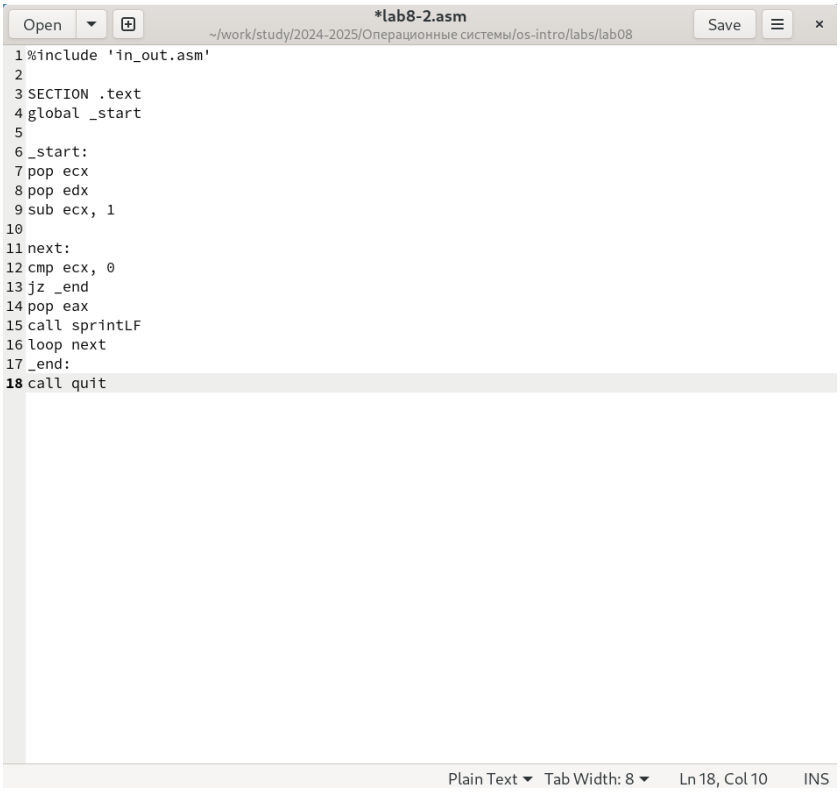

Рис. 8: программа из листинга

Я скомпилировал программу и запустил ее, указав аргументы. Программа обработала то же количество аргументов, что и было введено (Fig. -@fig:009).

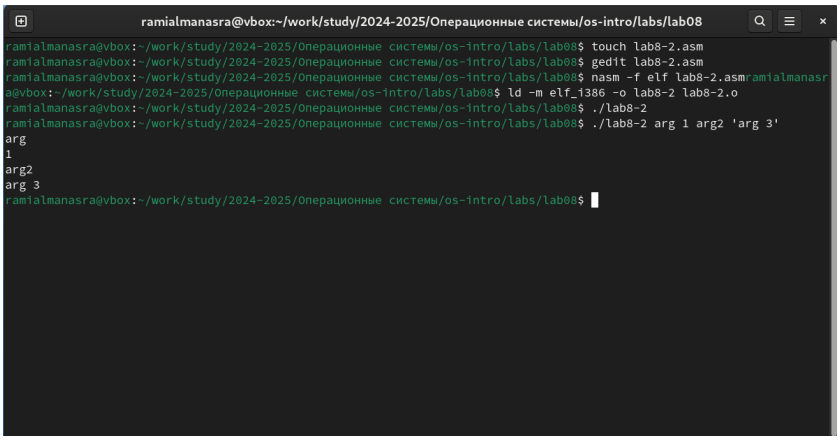

Рис. 9: запуск программы

Создаю новый файл для программы и копирую в него код из третьего листин-

га (Fig. -@fig:010).

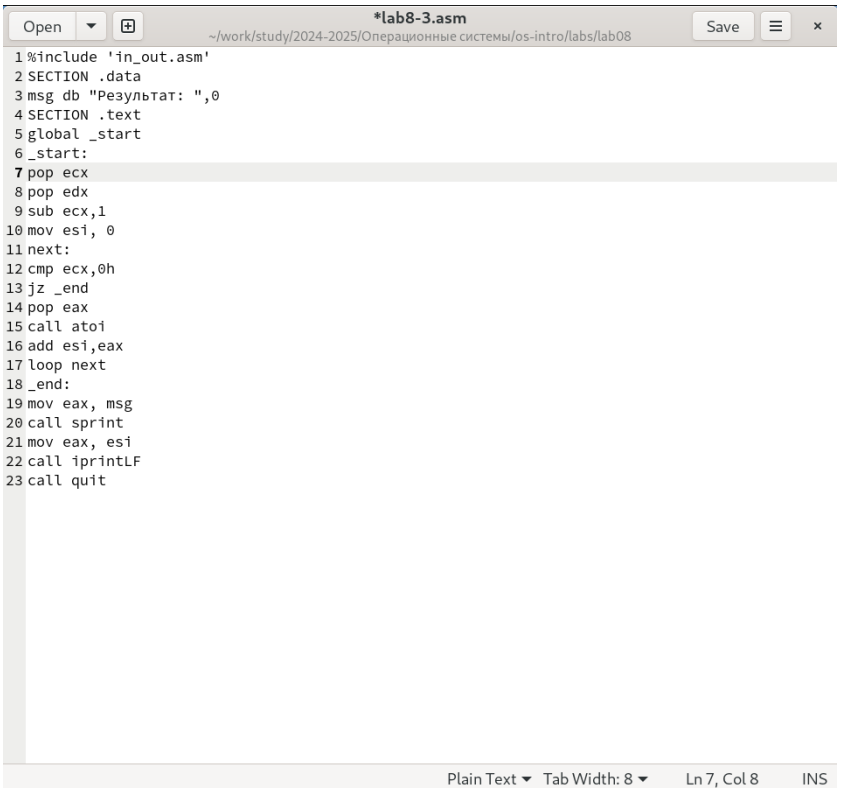

Рис. 10: программа из третьего листинга

Я компилирую программу и запускаю ее, указывая некоторые числа в качестве аргументов, программа суммирует их (Fig. -@fig:011).

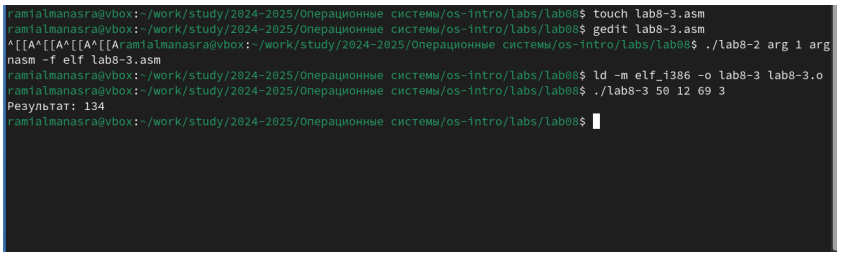

Рис. 11: запуск программы

Изменяю текст программы из листинга для вычисления произведения аргументов командной строки (Fig. -@fig:012).

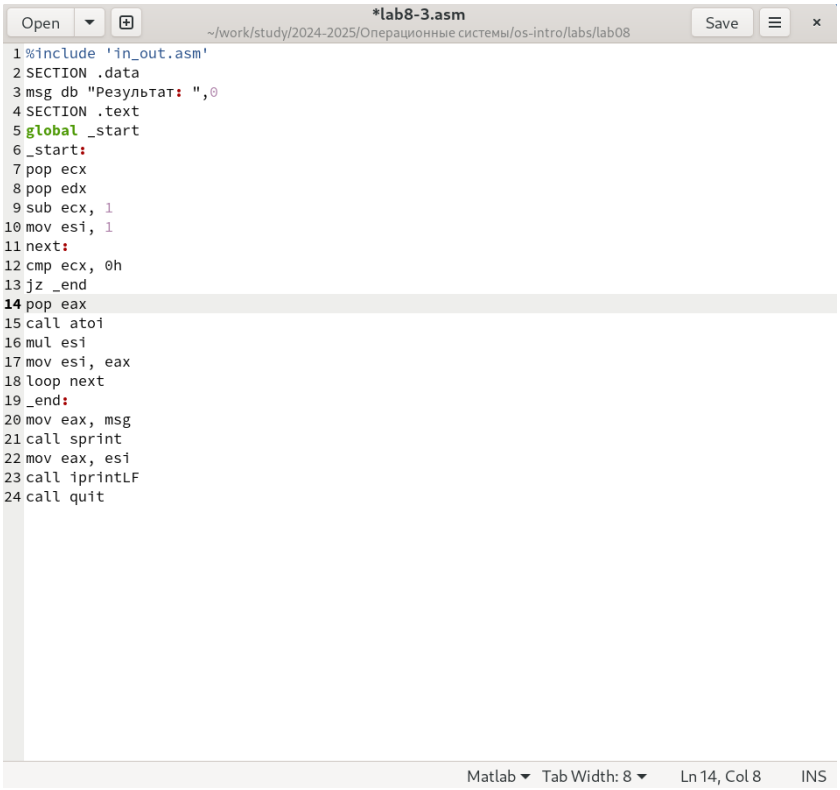

Рис. 12: Изменение текст программы

Теперь программа умножает введенные числа (Fig. -@fig:013).

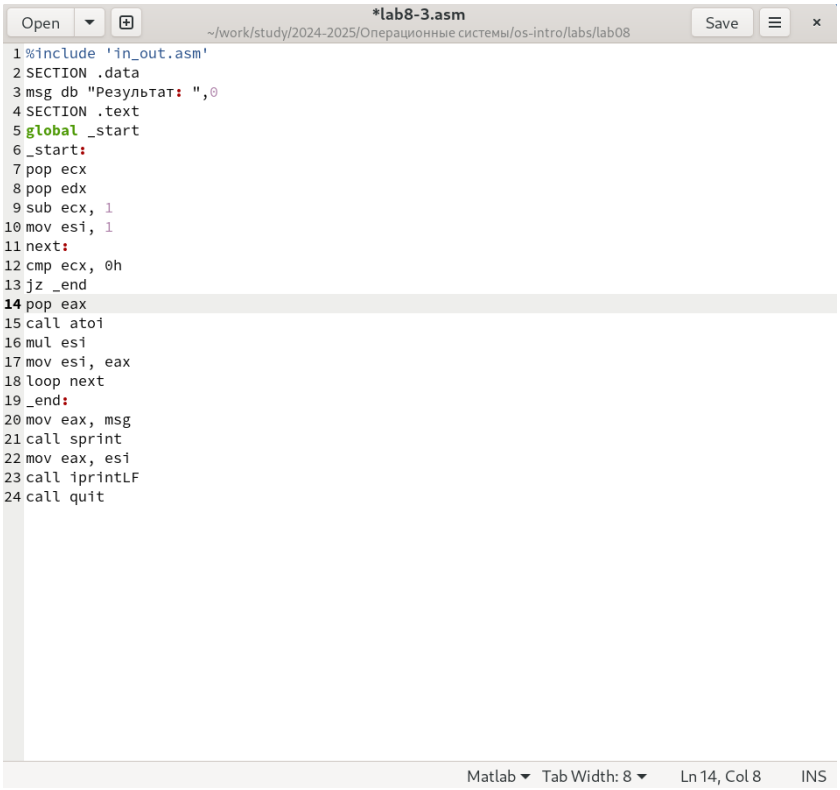

Рис. 13: запуск новой программы

Задание для самостоятельной работы

Я пишу программу, которая найдет сумму значений для функции f(x) = 10x-5, которая соответствует моему третьему варианту (Fig. -@fig:014).

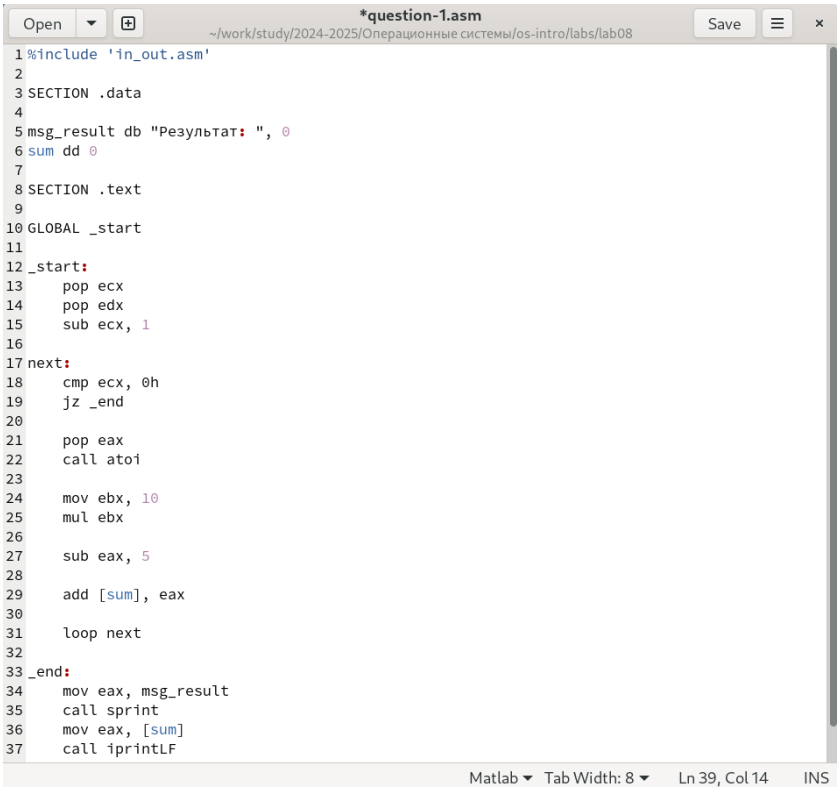

Рис. 14: программа, находящая сумму значений для функции f(x) = 10x-5

код программы:

```
%include 'in_out.asm'

SECTION .data

msg_result db "Результат: ", 0
sum dd 0

SECTION .text

GLOBAL _start
```

```
_start:
    pop ecx
    pop edx
    sub ecx, 1

next:
    cmp ecx, 0h
    jz _end

    pop eax
    call atoi

    mov ebx, 10
    mul ebx

    sub eax, 5

    add [sum], eax

    loop next

_end:
    mov eax, msg_result
    call sprint
    mov eax, [sum]
    call iprintLF

    call quit
```

Проверяю работу программы, указывая в качестве аргументов несколько чи-

сел (Fig. -@fig:015).

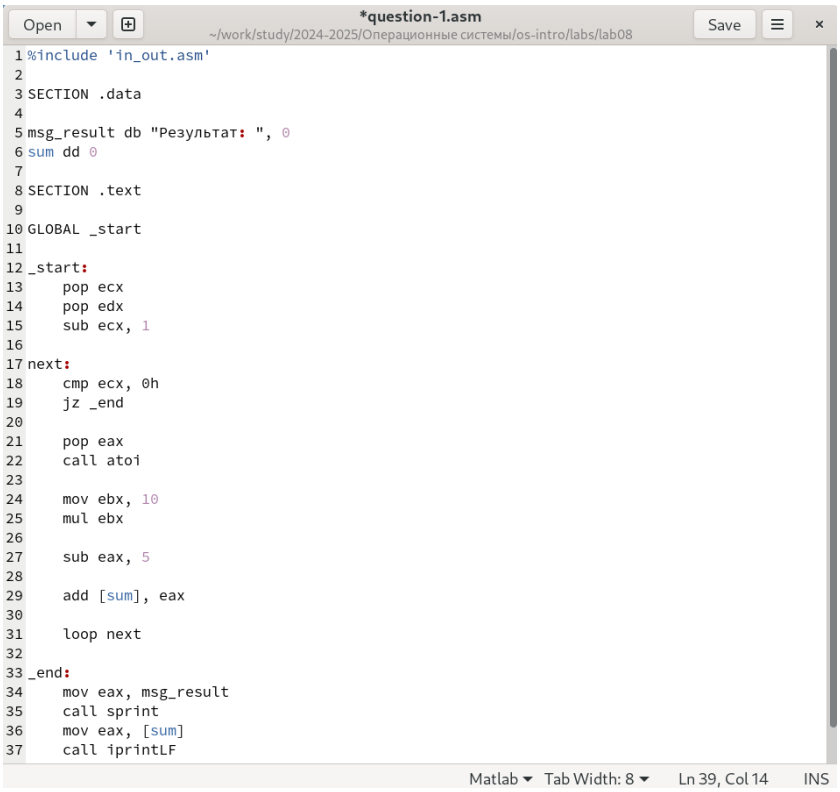

Рис. 15: запуск программы

Выводы

В результате этой лабораторной работы я приобрел навыки написания программ с использованием циклов, а также научился обрабатывать аргументы командной строки.

Список литературы

1. Course on TUIS
2. Laboratory Work No. 8
3. Programming in NASM Assembler Language, Stolyarov A. V.